

Computational Intelligence Laboratory

Sentiment Classification Project

Diego Rivera Garrido, David Wissel, Kai Lion

March 27th, 2026

Outline

- ▶ Introduction to the Task
- ▶ Overview of the Dataset
- ▶ Evaluation Metric
- ▶ NLP Concepts
- ▶ Model Architectures
- ▶ Tips

Section 1

Project on Sentiment Classification

Sentiment Classification

Given a product review, predict the number of stars the user gave for that product:

- ▶ Each review has a corresponding rating: integer from 0 to 4 (1-5 stars).

Sentiment Classification

Given a product review, predict the number of stars the user gave for that product:

- ▶ Each review has a corresponding rating: integer from 0 to 4 (1-5 stars).

Examples:

- ▶ **Positive:** *Must buy!*
- ▶ **Neutral:** *Not very sturdy but beautiful.*
- ▶ **Negative:** *This fitness tracker is the worst thing I've ever bought.*

Section 2

Overview of the Dataset

Dataset Provided

- ▶ The dataset was created from a ternary SC dataset using an undisclosed split percentage between train and test.
- ▶ **train.csv** ~ 252K labeled reviews

Dataset Provided

- ▶ The dataset was created from a ternary SC dataset using an undisclosed split percentage between train and test.
- ▶ **train.csv** ~ 252K labeled reviews
- ▶ **test.csv** ~168K unlabeled reviews
 - ▶ Kaggle's Public Leaderboard evaluates your submissions using only 45 % of the data
 - ▶ You are graded based on your score on the Private Leaderboard, which uses the remaining 55 % of the data.

Dataset Provided

- ▶ The dataset was created from a ternary SC dataset using an undisclosed split percentage between train and test.
- ▶ **train.csv** ~ 252K labeled reviews
- ▶ **test.csv** ~168K unlabeled reviews
 - ▶ Kaggle's Public Leaderboard evaluates your submissions using only 45 % of the data
 - ▶ You are graded based on your score on the Private Leaderboard, which uses the remaining 55 % of the data.

Tip: Use a local training/validation split (e.g. 90/10) or k-fold cross-validation to tune model hyperparameters. Do not validate on the Kaggle Leaderboard.

Dataset Description

Dataset Description

- ▶ Reviews consist of one “title” sentence, followed by a “paragraph”.

Dataset Description

- ▶ Reviews consist of one “title” sentence, followed by a “paragraph”.
- ▶ Reviews can be either in German or English (roughly 50-50 split).

Dataset Description

- ▶ Reviews consist of one “title” sentence, followed by a “paragraph”.
- ▶ Reviews can be either in German or English (roughly 50-50 split).
- ▶ No preprocessing was performed (white-spaces, stop words...).

Dataset Description

- ▶ Reviews consist of one “title” sentence, followed by a “paragraph”.
- ▶ Reviews can be either in German or English (roughly 50-50 split).
- ▶ No preprocessing was performed (white-spaces, stop words...).
- ▶ Expect typos, emojis, slang, repeated characters (e.g. sooooo), etc.

Dataset Description

- ▶ Reviews consist of one “title” sentence, followed by a “paragraph”.
- ▶ Reviews can be either in German or English (roughly 50-50 split).
- ▶ No preprocessing was performed (white-spaces, stop words...).
- ▶ Expect typos, emojis, slang, repeated characters (e.g. sooooo), etc.

Note: Adequate Data Pre-Processing can be useful.

Dataset - Food for Thought

Dataset - Food for Thought

- ▶ **Ambiguous Reviews:** *It took two weeks longer than expected to arrive, but it works great.*

Dataset - Food for Thought

- ▶ **Ambiguous Reviews:** *It took two weeks longer than expected to arrive, but it works great.*
- ▶ Sarcasm may exist in some reviews

Dataset - Food for Thought

- ▶ **Ambiguous Reviews:** *It took two weeks longer than expected to arrive, but it works great.*
- ▶ Sarcasm may exist in some reviews
- ▶ **Patterns indicative of label:** For example, ! can signal strong emotion (positive or negative), *...* indicates **bolded text**.

Dataset - Food for Thought

- ▶ **Ambiguous Reviews:** *It took two weeks longer than expected to arrive, but it works great.*
- ▶ Sarcasm may exist in some reviews
- ▶ **Patterns indicative of label:** For example, ! can signal strong emotion (positive or negative), *...* indicates **bolded text**.

Tip: Apply Domain Knowledge. Inspect and play around with your data!

Section 3

Evaluation Metric

Evaluation Metric

- ▶ **Sentiment Encoding:** 0,1,2,3,4
- ▶ **Scoring Function:**

$$L(\hat{\mathbf{y}}, \mathbf{y}) = 1 - \frac{1}{4n} \sum_{i=1}^n |y_i - \hat{y}_i| = 1 - \frac{\text{MAE}}{4},$$

where MAE is the mean absolute error between predictions and ground truth.

Evaluation Metric

- ▶ **Sentiment Encoding:** 0,1,2,3,4
- ▶ **Scoring Function:**

$$L(\hat{\mathbf{y}}, \mathbf{y}) = 1 - \frac{1}{4n} \sum_{i=1}^n |y_i - \hat{y}_i| = 1 - \frac{\text{MAE}}{4},$$

where MAE is the mean absolute error between predictions and ground truth.

- ▶ Severe misclassifications (0 stars $\leftrightarrow$ 4 stars) are penalized more severely than minor ones (e.g. 2 stars $\leftrightarrow$ 3 stars).

Baseline Scores

► Scoring Function:

$$L(\hat{\mathbf{y}}, \mathbf{y}) = 1 - \frac{1}{4n} \sum_{i=1}^n |y_i - \hat{y}_i| = 1 - \frac{\text{MAE}}{4},$$

where MAE is the mean absolute error between predictions and ground truth.

Baseline Scores

- ▶ **Scoring Function:**

$$L(\hat{\mathbf{y}}, \mathbf{y}) = 1 - \frac{1}{4n} \sum_{i=1}^n |y_i - \hat{y}_i| = 1 - \frac{\text{MAE}}{4},$$

where MAE is the mean absolute error between predictions and ground truth.

- ▶ The easy baseline for Grade 4 is set at a score of 0.875 on the public leaderboard.

Baseline Scores

- ▶ **Scoring Function:**

$$L(\hat{\mathbf{y}}, \mathbf{y}) = 1 - \frac{1}{4n} \sum_{i=1}^n |y_i - \hat{y}_i| = 1 - \frac{\text{MAE}}{4},$$

where MAE is the mean absolute error between predictions and ground truth.

- ▶ The easy baseline for Grade 4 is set at a score of 0.875 on the public leaderboard.
- ▶ The hard baseline for Grade 6 is set at a score of 0.906 on the public leaderboard.

Section 4

NLP concepts

Intro to Tokenization

In machine learning, we typically have:

$$A\mathbf{x} = \mathbf{y}$$

where $\mathbf{x}$ is our input vector.

What happens when $\mathbf{x}$ is a sentence?

Tokenization

- ▶ **Tokenization** splits text into smaller units (i.e. tokens) that can then be represented numerically.
- ▶ Example sentence: "I can't believe it's already Friday!"
- ▶ Tokenization:
["I", "can't", "believe", "it's", "already", "Friday!"]

Tokenization

- ▶ **Tokenization** splits text into smaller units (i.e. tokens) that can then be represented numerically.
- ▶ Example sentence: “I can’t believe it’s already Friday!”
- ▶ Tokenization:
[“I”, “can’t”, “believe”, “it’s”, “already”, “Friday!”]
- ▶ Another tokenization:
[“I”, “ can”, “ ’t”, “ ”, “believe”, “it’s”, “ al”, “ready ”, “Fri”, “day”, “!”]

Tokenization - Building a Vocabulary

- ▶ First, we tokenize the texts in our data.
- ▶ Then, we build a **vocabulary** of the tokens found in the dataset.

Tokenization - Building a Vocabulary

- ▶ First, we tokenize the texts in our data.
- ▶ Then, we build a **vocabulary** of the tokens found in the dataset.
- ▶ Each token is represented by an index in the dataset.
[“I”, “can’t”, “believe”, “it’s”, “already”, “Friday!”]
[0, 1, 2, 3, 4, 5]

Vectorization

- ▶ Each token should then be **vectorized**, i.e. assigned to a real-valued vector $\mathbf{x}_{w_i}$.

Vectorization

- ▶ Each token should then be **vectorized**, i.e. assigned to a real-valued vector $\mathbf{x}_{w_i}$.
- ▶ In other words, it is represented by an **embedding** in d -dimensional space, $\mathbf{x}_{w_i} \in \mathbb{R}^d$.

Vectorization

- ▶ Each token should then be **vectorized**, i.e. assigned to a real-valued vector $\mathbf{x}_{w_i}$.
- ▶ In other words, it is represented by an **embedding** in d -dimensional space, $\mathbf{x}_{w_i} \in \mathbb{R}^d$.
- ▶ This vector should capture the meaning of the token (c.f. Hinton's **Thought vectors**).

Vectorization

- ▶ Each token should then be **vectorized**, i.e. assigned to a real-valued vector $\mathbf{x}_{w_i}$.
- ▶ In other words, it is represented by an **embedding** in d -dimensional space, $\mathbf{x}_{w_i} \in \mathbb{R}^d$.
- ▶ This vector should capture the meaning of the token (c.f. Hinton's **Thought vectors**).
- ▶ Tokens that have similar semantic meaning should have similar embeddings (e.g. large $\langle \mathbf{x}_{w_i}, \mathbf{x}_{w_j} \rangle$).

Bag-of-Words Vectorization

- ▶ Bag-of-Words (BoW) is a naive vectorization method
- ▶ It counts how many times each word appears in the text.

Bag-of-Words Vectorization

- ▶ Bag-of-Words (BoW) is a naive vectorization method
- ▶ It counts how many times each word appears in the text.
- ▶ Example:
 - Sentence: "I love CIL."
 - Vocabulary: ["I", "and", "to", "love", "CIL"]
 - Vectorization: [1, 0, 0, 1, 1]

Bag-of-Words Vectorization

- ▶ Bag-of-Words (BoW) is a naive vectorization method
- ▶ It counts how many times each word appears in the text.
- ▶ Example:
 - Sentence: "I love CIL."
 - Vocabulary: ["I", "and", "to", "love", "CIL"]
 - Vectorization: [1, 0, 0, 1, 1]
- ▶ This vectorization does **not** capture the similarity between words .

Other Examples of Vectorizations

Other Examples of Vectorizations

- ▶ GloVe: co-occurrence matrix - number of occurrences of word w_i in the context of word w_j

Other Examples of Vectorizations

- ▶ GloVe: co-occurrence matrix - number of occurrences of word w_i in the context of word w_j
- ▶ Word2Vec, FastText and GloVe are all pre-trained word vectorizations trained on massive text corpora.

Other Examples of Vectorizations

- ▶ GloVe: co-occurrence matrix - number of occurrences of word w_i in the context of word w_j
- ▶ Word2Vec, FastText and GloVe are all pre-trained word vectorizations trained on massive text corpora.
- ▶ TF-IDF (term frequency–inverse document frequency): higher importance to words that are frequent in a document but rare across other documents.

Other Examples of Vectorizations

- ▶ GloVe: co-occurrence matrix - number of occurrences of word w_i in the context of word w_j
- ▶ Word2Vec, FastText and GloVe are all pre-trained word vectorizations trained on massive text corpora.
- ▶ TF-IDF (term frequency–inverse document frequency): higher importance to words that are frequent in a document but rare across other documents.
- ▶ `nn.Embedding` in PyTorch: maps discrete input tokens to dense vectors which are learned during **training**.

Other Examples of Vectorizations

- ▶ GloVe: co-occurrence matrix - number of occurrences of word w_i in the context of word w_j
- ▶ Word2Vec, FastText and GloVe are all pre-trained word vectorizations trained on massive text corpora.
- ▶ TF-IDF (term frequency–inverse document frequency): higher importance to words that are frequent in a document but rare across other documents.
- ▶ `nn.Embedding` in PyTorch: maps discrete input tokens to dense vectors which are learned during **training**.

SOTA embeddings are both multi-layered and take the context of the word in the text (in our case the review sentence) into account (e.g. BERT, GPT).

Section 5

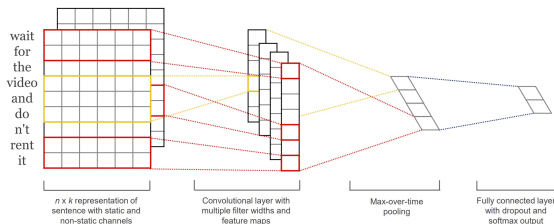
Model Architectures

Simple Baselines

- ▶ Simple Vectorization (for example Bag-of-Words, GloVe, TF-IDF, etc.)
- ▶ Train a Linear Classifier on top of it
- ▶ Tip: Highly Recommend implementing these baselines using existing libraries (e.g. `sklearn`).

Neural Network Baselines

- ▶ MLP on top of embeddings
- ▶ 1D Convolutional neural networks: captures local dependencies.
 - ▶ Use pooling to get sentence-wise embedding.
 - ▶ Add a final layer fully-connected layer for predictions.



Source: Yoon Kim 2014

RNNs

- ▶ Recurrent Neural Networks (RNNs) process sequences token by token, maintaining memory of previous inputs.
- ▶ They allow **context** to be taken into account.

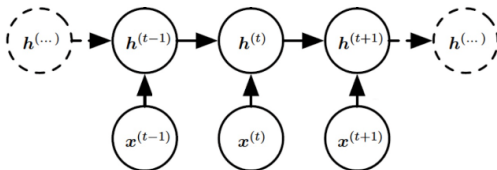


Figure: RNN over a finite horizon

Source: Machine Perception Lecture Notes by O. Hilliges

RNNs

- ▶ Recurrent Neural Networks (RNNs) process sequences token by token, maintaining memory of previous inputs.
- ▶ They allow **context** to be taken into account.

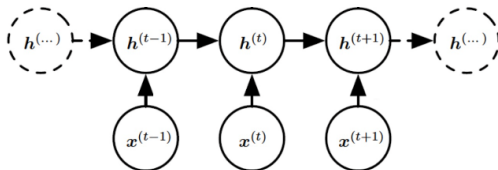


Figure: RNN over a finite horizon

Source: Machine Perception Lecture Notes by O. Hilliges

- ▶ **Bidirectional RNNs** read the sequence both forwards and backwards.
- ▶ This captures richer context, especially useful when meaning depends on both past and future information.

Transformer Models

- ▶ Self-attention layers in Transformer models allow each token to attend simultaneously to every token in the sentence.
- ▶ The sentence is processed through multiple attention layers.

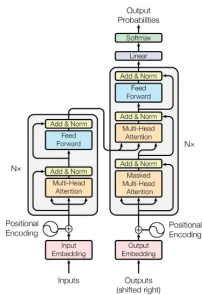


Figure: Transformer Architecture

Transformer Models

- ▶ Transformer Models are data hungry!

Transformer Models

- ▶ Transformer Models are data hungry!
- ▶ Our dataset (252k reviews) may be small, the model risks overfitting.

Transformer Models

- ▶ Transformer Models are data hungry!
- ▶ Our dataset (252k reviews) may be small, the model risks overfitting.
- ▶ Tip: Instead of training from scratch, better to fine-tune the model

Transformer Models

- ▶ Transformer Models are data hungry!
- ▶ Our dataset (252k reviews) may be small, the model risks overfitting.
- ▶ Tip: Instead of training from scratch, better to fine-tune the model
 - ▶ Freezing Some Layers
 - ▶ LoRA
 - ▶ etc

Transformer Models

- ▶ The use of pre-trained models is allowed.

Transformer Models

- ▶ The use of pre-trained models is allowed.
- ▶ The models should be **publicly** available.

Transformer Models

- ▶ The use of pre-trained models is allowed.
- ▶ The models should be **publicly** available.
- ▶ You should not adopt ready-to-use external APIs (e.g. ChatGPT API).

Transformer Models

- ▶ The use of pre-trained models is allowed.
- ▶ The models should be **publicly** available.
- ▶ You should not adopt ready-to-use external APIs (e.g. ChatGPT API).
- ▶ Tip: Use the huggingface transformers library
<https://github.com/huggingface/transformers>.

Recap of Potential Model Architectures

- ▶ Linear Models
- ▶ MLPs
- ▶ 1D CNNs
- ▶ (Bidirectional) RNNs
- ▶ Transformer Models
- ▶ etc.

Section 6

Tips

Tips

Tips

- ▶ Creativity matters (report and ideas are 50% of the grade)!

Tips

- ▶ Creativity matters (report and ideas are 50% of the grade)!
- ▶ Instead of focusing too much on the leaderboard score, come up with elegant tokenizations, model architectures, etc.

Tips

- ▶ Creativity matters (report and ideas are 50% of the grade)!
- ▶ Instead of focusing too much on the leaderboard score, come up with elegant tokenizations, model architectures, etc.
- ▶ Methodological Interpretation of Results and Failures

Tips

- ▶ Creativity matters (report and ideas are 50% of the grade)!
- ▶ Instead of focusing too much on the leaderboard score, come up with elegant tokenizations, model architectures, etc.
- ▶ Methodological Interpretation of Results and Failures
- ▶ Scientific rigor is valued, compare your method against standard/baseline methods (ablation study)

Tips

- ▶ Creativity matters (report and ideas are 50% of the grade)!
- ▶ Instead of focusing too much on the leaderboard score, come up with elegant tokenizations, model architectures, etc.
- ▶ Methodological Interpretation of Results and Failures
- ▶ Scientific rigor is valued, compare your method against standard/baseline methods (ablation study)
- ▶ Start with simple Baseline Models: Linear Models, CNNs. Then, move to more complex ones: RNNs, etc.

Tips

- ▶ Creativity matters (report and ideas are 50% of the grade)!
- ▶ Instead of focusing too much on the leaderboard score, come up with elegant tokenizations, model architectures, etc.
- ▶ Methodological Interpretation of Results and Failures
- ▶ Scientific rigor is valued, compare your method against standard/baseline methods (ablation study)

- ▶ Start with simple Baseline Models: Linear Models, CNNs. Then, move to more complex ones: RNNs, etc.
- ▶ Use of complex models should be justified both quantitatively (i.e. by the score) and practically (10x computation for 0.1 accuracy gain not worth it)

Tips

- ▶ Creativity matters (report and ideas are 50% of the grade)!
- ▶ Instead of focusing too much on the leaderboard score, come up with elegant tokenizations, model architectures, etc.
- ▶ Methodological Interpretation of Results and Failures
- ▶ Scientific rigor is valued, compare your method against standard/baseline methods (ablation study)

- ▶ Start with simple Baseline Models: Linear Models, CNNs. Then, move to more complex ones: RNNs, etc.
- ▶ Use of complex models should be justified both quantitatively (i.e. by the score) and practically (10x computation for 0.1 accuracy gain not worth it)
- ▶ Have fun and enjoy yourselves!

Tips

More submissions doesn't necessarily mean a better result, don't overfit to the public leaderboard (validate locally!):

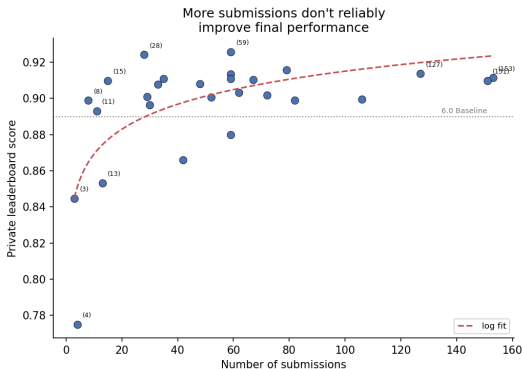


Figure: Number of submissions vs. private leaderboard score

Sample implementation

- ▶ Very Naive Baseline: Bag-of-Words Vectorizer + Logistic Regression